LABORATORIO 8.1: Navegación Dinámica con Paso de Argumento

1. Contexto y Objetivos de Aprendizaje

Contexto: En aplicaciones del mundo real, como sistemas de gestión de eventos o redes sociales, es fundamental el patrón *Master-Detail*. Al seleccionar un elemento de una lista, la aplicación debe saber exactamente qué datos mostrar en la siguiente pantalla. Gestionar esto mediante argumentos en la URL de navegación asegura que el flujo sea predecible y fácil de depurar.

Objetivos:

1. Configurar rutas de navegación dinámicas con parámetros múltiples.
2. Recuperar y procesar argumentos desde el NavBackStackEntry.
3. Implementar componentes de **Material 3 (Card)** para visualización de datos con elevación.

Nota:

Este laboratorio planteado está diseñado bajo una lógica de **continuidad pedagógica**. Esto significa que:

- **Parte de SafePass 2026:** El estudiante no empieza desde cero este laboratorio. Utiliza el "proyecto estandarizado" creado en el **Laboratorio 7.2**.
- **Aprendizaje Incremental:** En el Lab 7.2 construyeron la base, en el Lab 8.1 crearon el flujo de Login, y en este Lab aprenderán a pasar datos dinámicos entre las pantallas ya existentes.

2. Requisitos Previos

- Tener finalizado el **Laboratorio 7.2** (Estructura base del proyecto SafePass 2026).
- Tener finalizado el **Laboratorio 8.1** (Configuración inicial de NavHost y navegación básica).
- La librería androidx.navigation:navigation-compose debe estar sincronizada en el build.gradle.kts.

3. Descripción General

Los estudiantes modificarán la lista de asistentes creada anteriormente para que cada elemento sea "clicleable". Al interactuar, la App navegará a una nueva pantalla de detalle pasando el ID y el Nombre del asistente a través de la ruta. En la pantalla de destino, se recuperarán estos valores para mostrarlos en una tarjeta elegante.

4. Paso a Paso Detallado

Paso 1: Definición del Modelo de Datos (Verificación)

Asegurarse de que su modelo de datos sea inmutable, tal como se revisó en la Semana 3 de clases.

1. **Ubicación:** app/src/main/java/com/puce/safepass/models/Asistente.kt.
2. **Código:** Si no existe, crearlo; si existe, verificar que sea una data class.

```
package com.puce.safepass.models

// POR QUÉ: Usamos data class para aprovechar la inmutabilidad y el método copy()
data class Asistente(
    val id: String,
    val nombre: String,
    val tipo: String
)
```

Paso 2: Creación de la Pantalla de Detalle (DetailScreen)

Esta pantalla recibirá los datos y los mostrará en una Card de Material 3.

1. **Ruta:** Hacer clic derecho en la carpeta ui.screens -> New -> Kotlin File. Nombrarlo DetailScreen.
2. **Código:** Agregar el siguiente contenido:

```
package com.puce.safepass.ui.screens

import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp

@Composable
fun DetailScreen(asistenteId: String?, asistenteNombre: String?) {
    Column(
        modifier = Modifier.fillMaxSize().padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        // POR QUÉ: Elevación y contenedores son estándar en Material 3
        Card(
            modifier = Modifier.fillMaxWidth().padding(16.dp),
            elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
        ) {
            Column(modifier = Modifier.padding(24.dp)) {
                Text(text = "ID de Asistente: ${asistenteId ?: "N/A"}", style =
MaterialTheme.typography.labelLarge)
                Spacer(modifier = Modifier.height(8.dp))
                Text(text = "Nombre: ${asistenteNombre ?: "Desconocido"}", style =
MaterialTheme.typography.headlineMedium)
            }
        }
```

```
    }
}
```

Paso 3: Modificación de la Lista de Asistentes

Haremos que la lista sea interactiva para disparar la navegación.

1. **Ruta:** Abrir app/src/main/java/com/puce/safepass/ui/screens/AssistantListScreen.kt.
2. **Modificación:** En el LazyColumn, dentro del ítem de la lista, agrega el modificador clickable.

```
// Fragmento de código para agregar/reemplazar dentro de la lista
ListItem(
    headlineContent = { Text(asistente.nombre) },
    modifier = Modifier.clickable {
        // Esta lambda se definirá en el NavGraph
        onAsistenteClick(asistente.id, asistente.nombre)
    }
)
```

Paso 4: Configuración de Argumentos en el NavGraph

Aquí definimos la ruta dinámica con el patrón detail/{id}/{nombre}.

1. **Ubicación:** app/src/main/java/com/puce/safepass/ui/navigation/NavGraph.kt.
2. **Modificación:** Reemplazar el bloque NavHost para incluir el nuevo destino con argumentos.

```
NavHost(navController = navController, startDestination = "list") {

    composable("list") {
        AssistantListScreen(onAsistenteClick = { id, nombre ->
            // Navegación con argumentos dinámicos
            navController.navigate("detail/$id/$nombre")
        })
    }

    // POR QUÉ: Definimos parámetros en la ruta entre llaves {}
    composable("detail/{asistenteId}/{asistenteNombre}") { backStackEntry ->
        // Recuperamos los argumentos del Bundle de navegación
        val id = backStackEntry.arguments?.getString("asistenteId")
        val nombre = backStackEntry.arguments?.getString("asistenteNombre")

        DetailScreen(asistenteId = id, asistenteNombre = nombre)
    }
}
```

5. Resultados Esperados (Verificación)

1. **Compilación:** Presionar el ícono del martillo (*Build*) y verificar que no existan errores de tipos.
2. **Ejecución:** Correr la app en el emulador (Shift + F10).
3. **Flujo Dinámico:** Al hacer clic en un asistente llamado "Ana", la nueva pantalla debe mostrar en una tarjeta: **"Nombre: Ana"**.
4. **Persistencia:** Si rota la pantalla del emulador, los datos del detalle deben permanecer visibles (el backStackEntry sobrevive a la recreación).

6. Desafío Extra

Reto: Hacer que la ruta de detalle sea más robusta. Si el nombre no se pasa (es opcional), la aplicación debe mostrar "Usuario Invitado" por defecto usando el **Operador Elvis (?:)** de Kotlin visto en la Semana 2.

Resolución:

1. Modificar la ruta en el NavGraph: detail/{asistenteId}?nombre={asistenteNombre}.
2. En DetailScreen, usar: Text(text = asistenteNombre ?: "Usuario Invitado").

7. Entregables

- Informe PDF con:
  - **Captura de código:** Copia del archivo NavGraph.kt.
  - **Evidencia de:** capturas del emulador donde:
    1. El emulador muestre la tarjeta de detalle con los datos de un asistente específico.
    2. Demás puntos que debe contener el informe como conclusiones, recomendaciones, etc.

8. Criterios de Evaluación

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe